Latent spaces and autoencoders

SEMTM0051

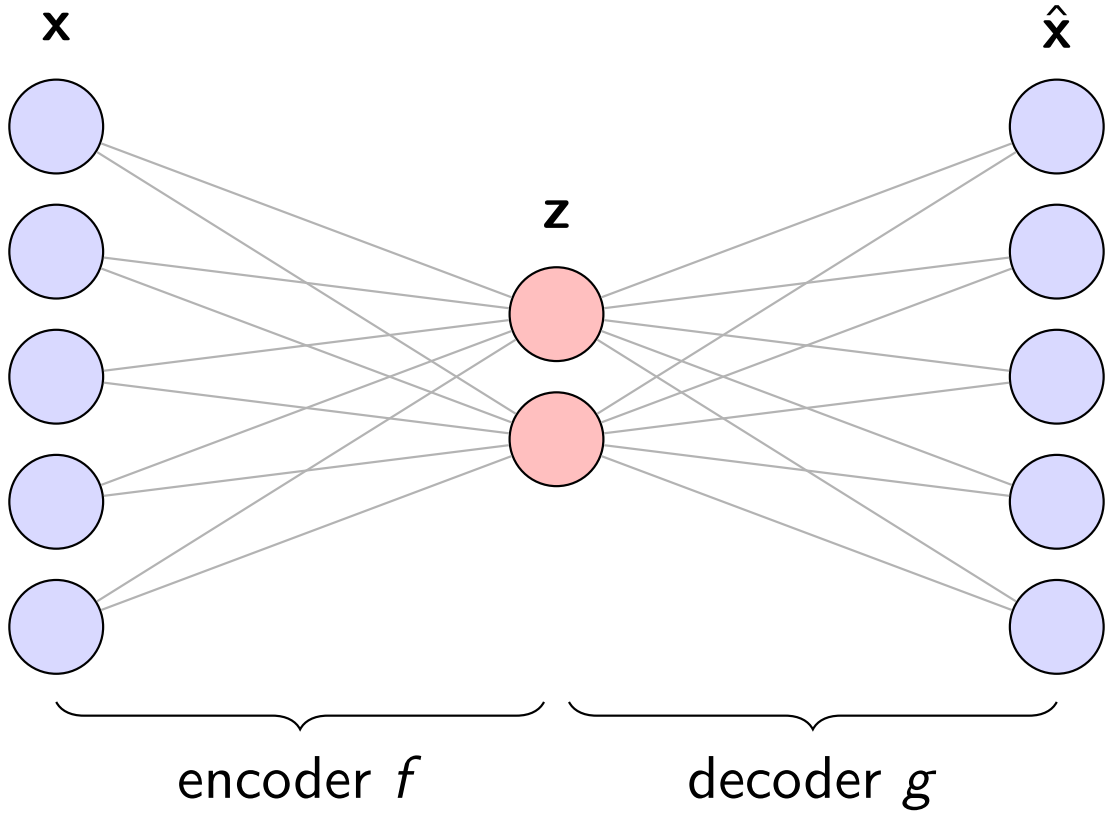
SEMT, U Bristol / `semtm0051.github.io`

2026-11

Motivation

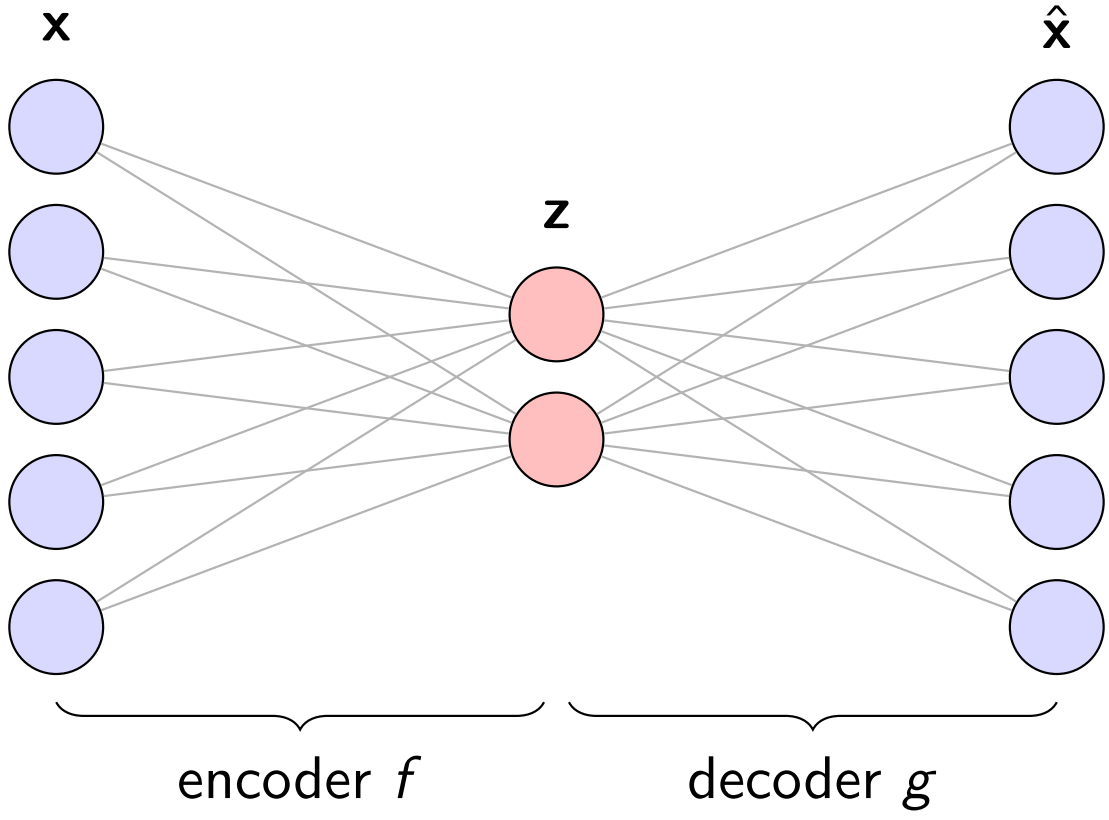
Today we will look at *auto-encoders*; auto-encoders are sometimes useful in and of themselves, however we are studying them as part of our effort to understand what makes networks work and because they provide a language or description of what networks are doing.

Auto-encoder



Train so that $\hat{\mathbf{x}} = g(f(\mathbf{x})) \approx \mathbf{x}$, for example by minimising $\|\mathbf{x} - \hat{\mathbf{x}}\|^2$.

Latent space



The space in the middle is called the *latent space*.

Motivation

original $\rightarrow$ latent representation $\rightarrow$ reconstruction

The latent space should 'pick out' what's important about the original so that it can reconstruct it even if the latent representation lives in a lower dimension!

MNIST: origin

In the late 1980s LeCun and colleagues at Bell Labs were building networks to read ZIP codes on US mail.

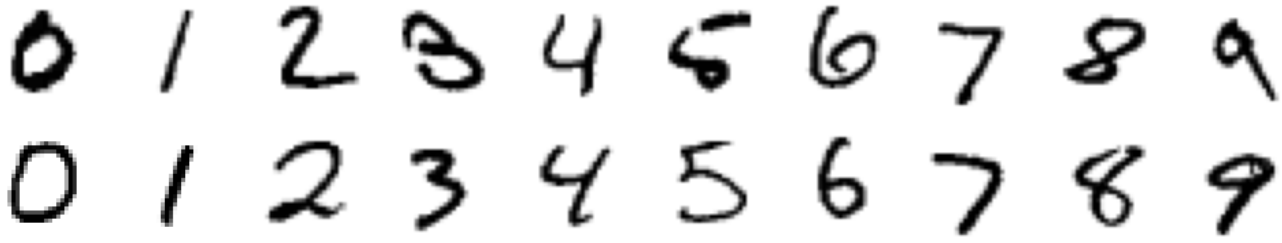
MNIST (1998) was built from two older NIST databases of handwritten digits:

- one written by US Census Bureau employees,
- one written by high-school students.

NIST had used the first for training and the second for testing, but the students' writing was harder to read, so MNIST mixes the two in both training and test sets: the "M" is for "modified".

MNIST: the data

- 60 000 training images and 10 000 test images.
- Each is 28×28 greyscale, a vector in $\mathbb{R}^{784}$.
- Each has a label $0, \ldots, 9$.



It is the standard beginner data set, and it is useful because the structure is simple: ten discrete categories plus noise, the noise being the variability of human handwriting.

MNIST: a caution

It is too easy.

- Linear classifier: about 92%.
- Small multilayer perceptron: about 98%.
- Good convolutional network: about 99.7%.

The last few errors are mostly ambiguous or mislabelled digits, so comparing good networks on MNIST probes the idiosyncrasies of the labelling, not the networks.

For us that is fine: we want to see what a network is doing, not to win.

An autoencoder for MNIST

$$784 \to 512 \to 256 \to 2 \to 256 \to 512 \to 784$$

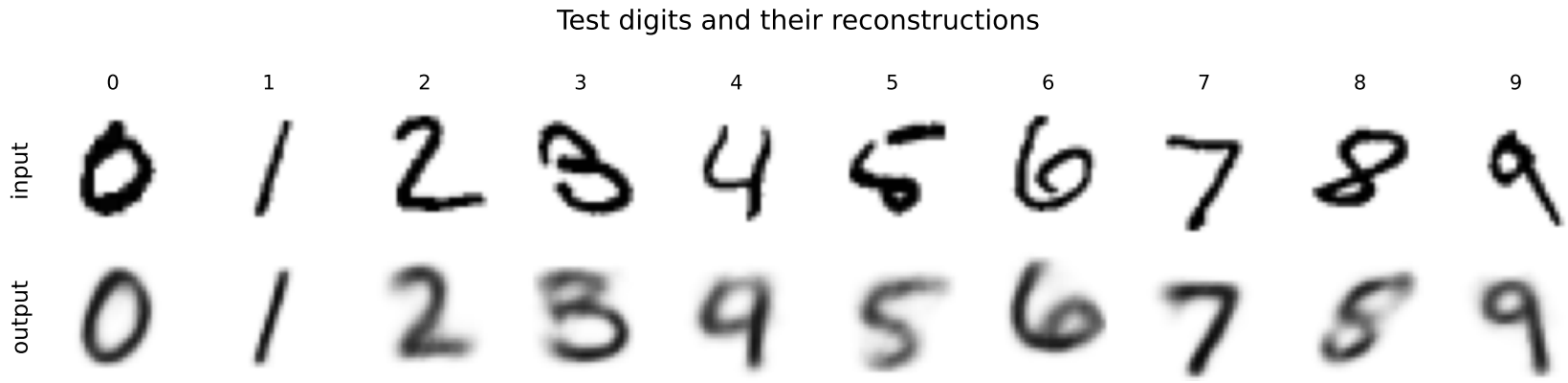
An autoencoder for MNIST

$$784 \to 512 \to 256 \to 2 \to 256 \to 512 \to 784$$

- Hidden layers: ReLU, $\phi(u) = \max(0, u)$.
- Bottleneck: linear, so $\mathbf{z} \in \mathbb{R}^2$.
- Output: sigmoid, so $\hat{x}_i \in [0, 1]$.
- Loss: binary cross-entropy,

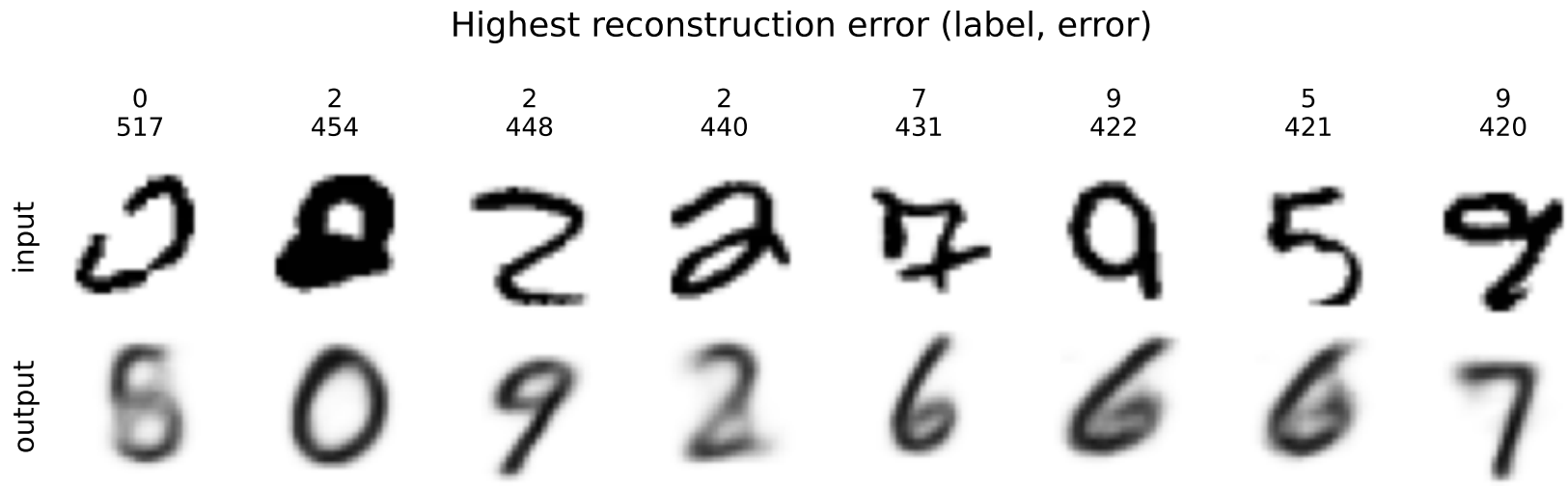
$$L = -\sum_{i=1}^{784} \left[x_i \log \hat{x}_i + (1 - x_i) \log(1 - \hat{x}_i) \right].$$

Reconstructions



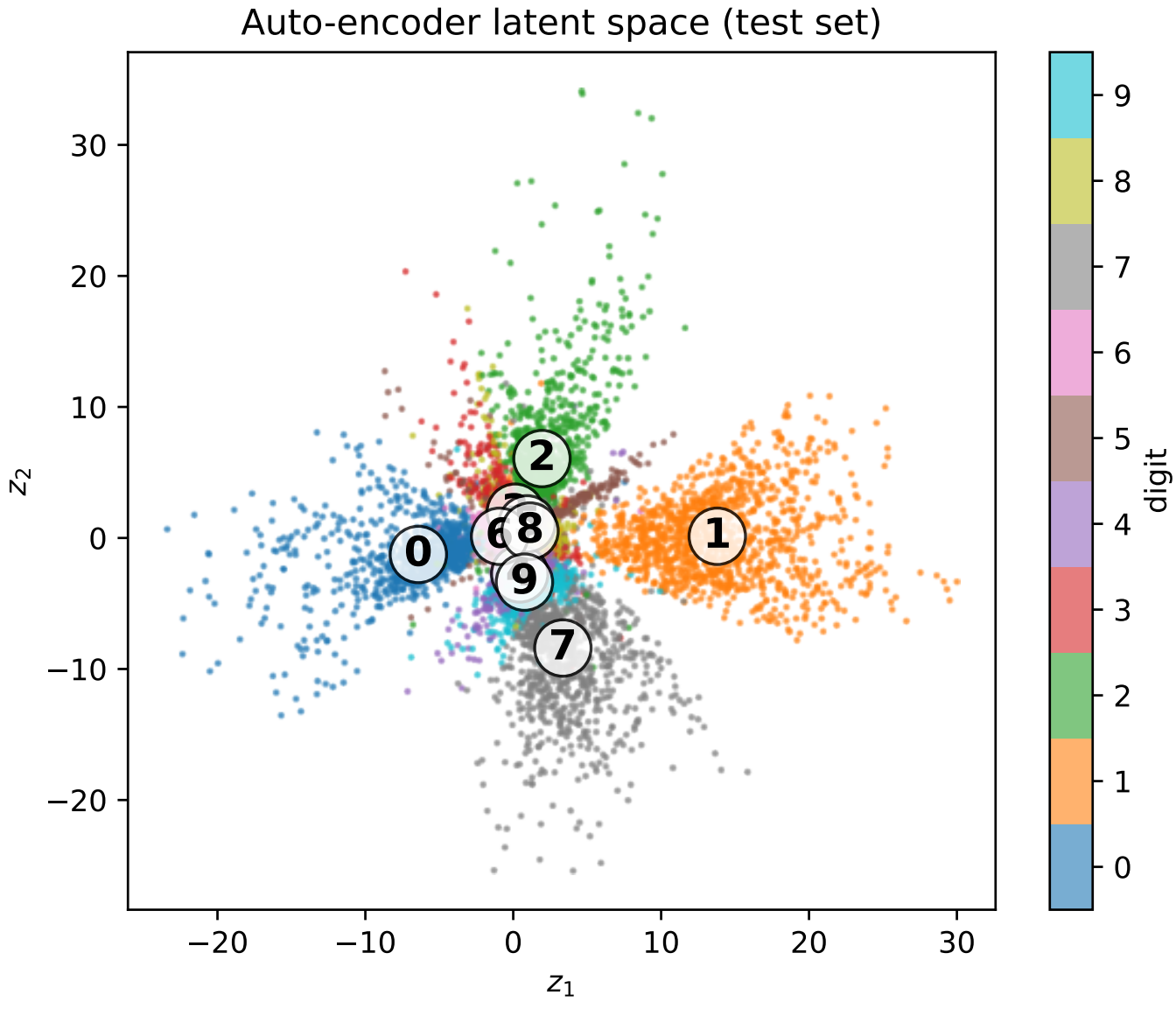
Each digit is squeezed through two numbers and expanded back out again; the reconstructions are blurred but recognisable.

The worst reconstructions

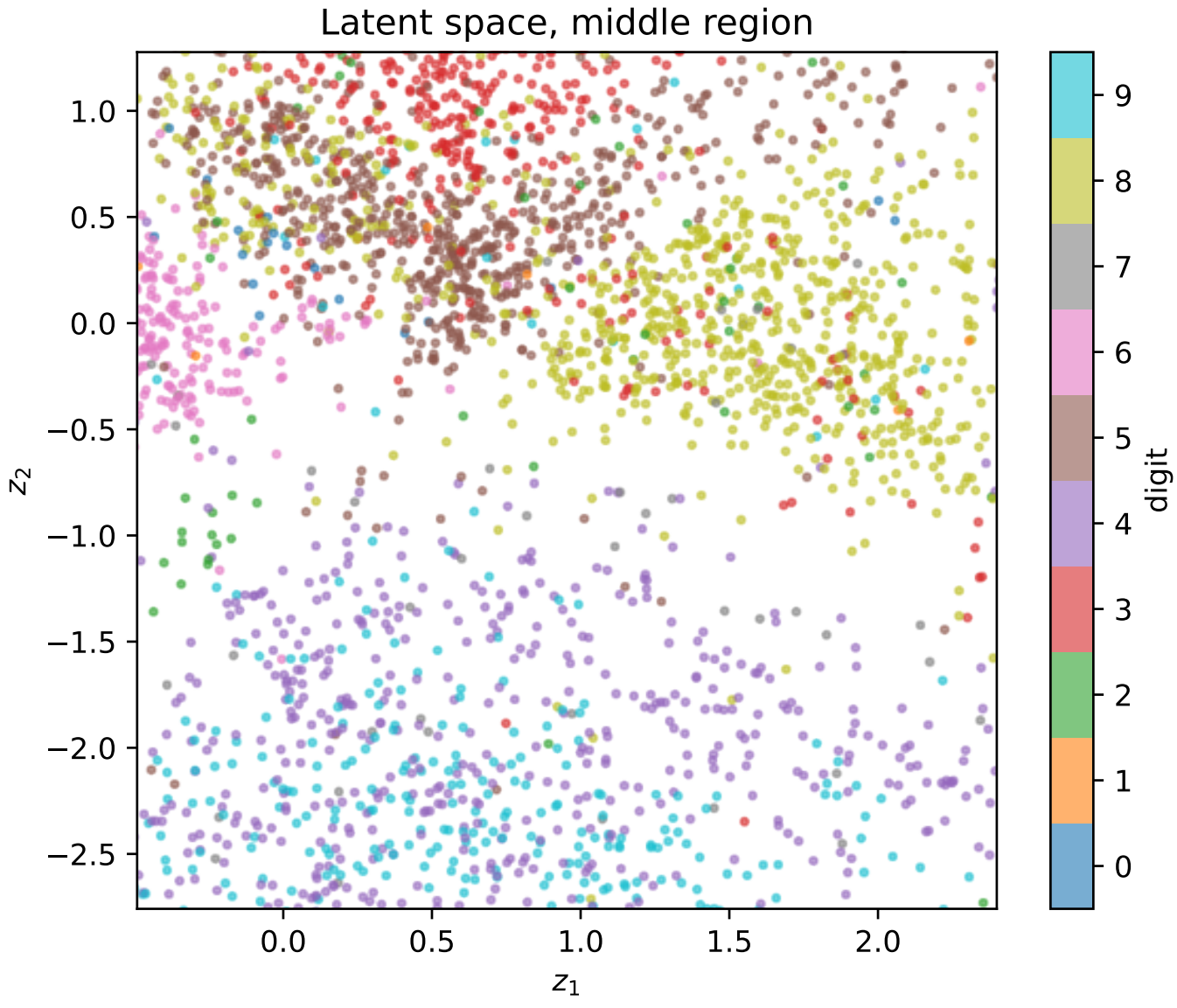


The test digits with the highest reconstruction error: mostly unusual or badly written digits, which the decoder replaces with something more typical.

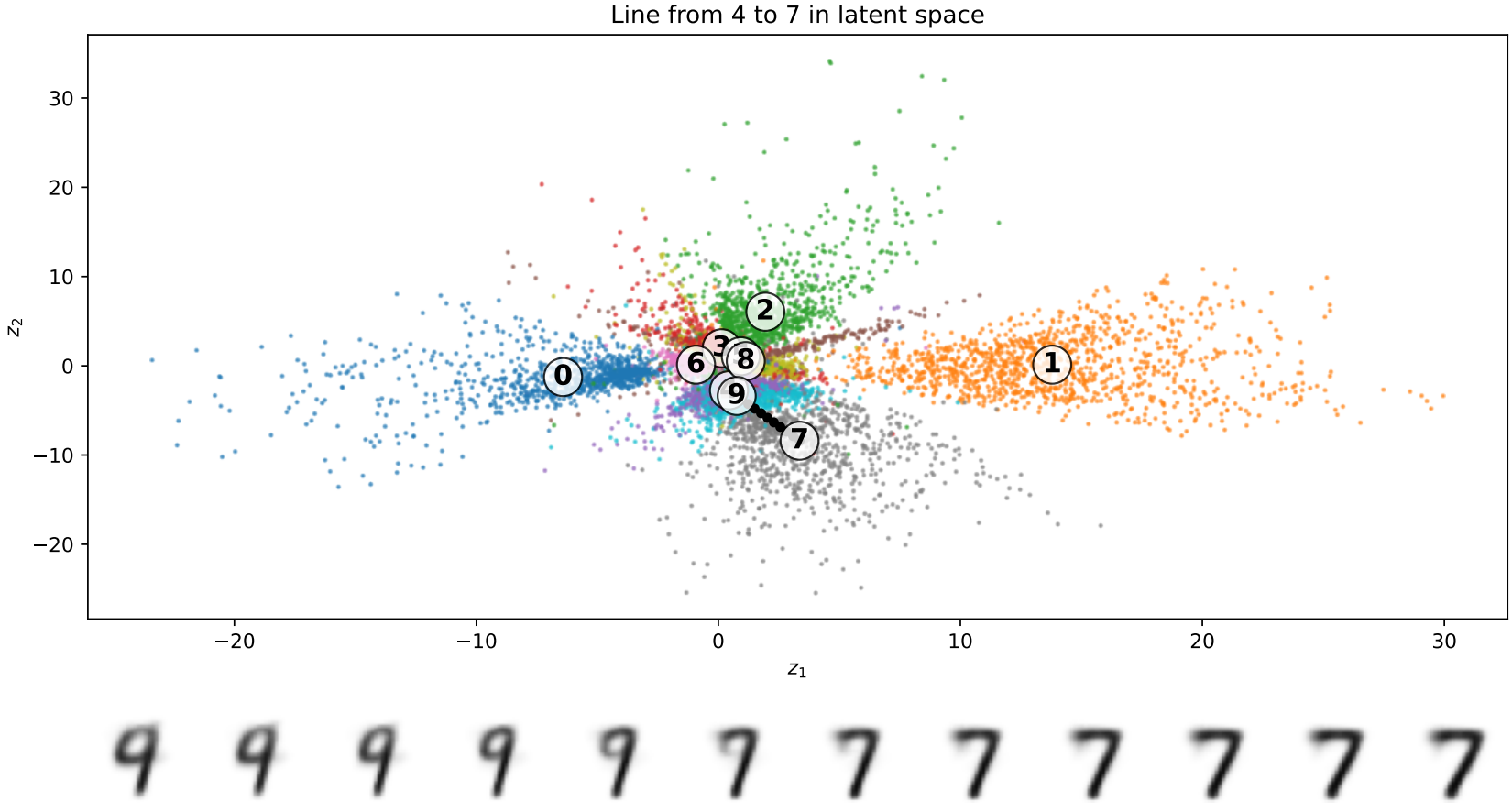
The latent space



The latent space, zoomed in

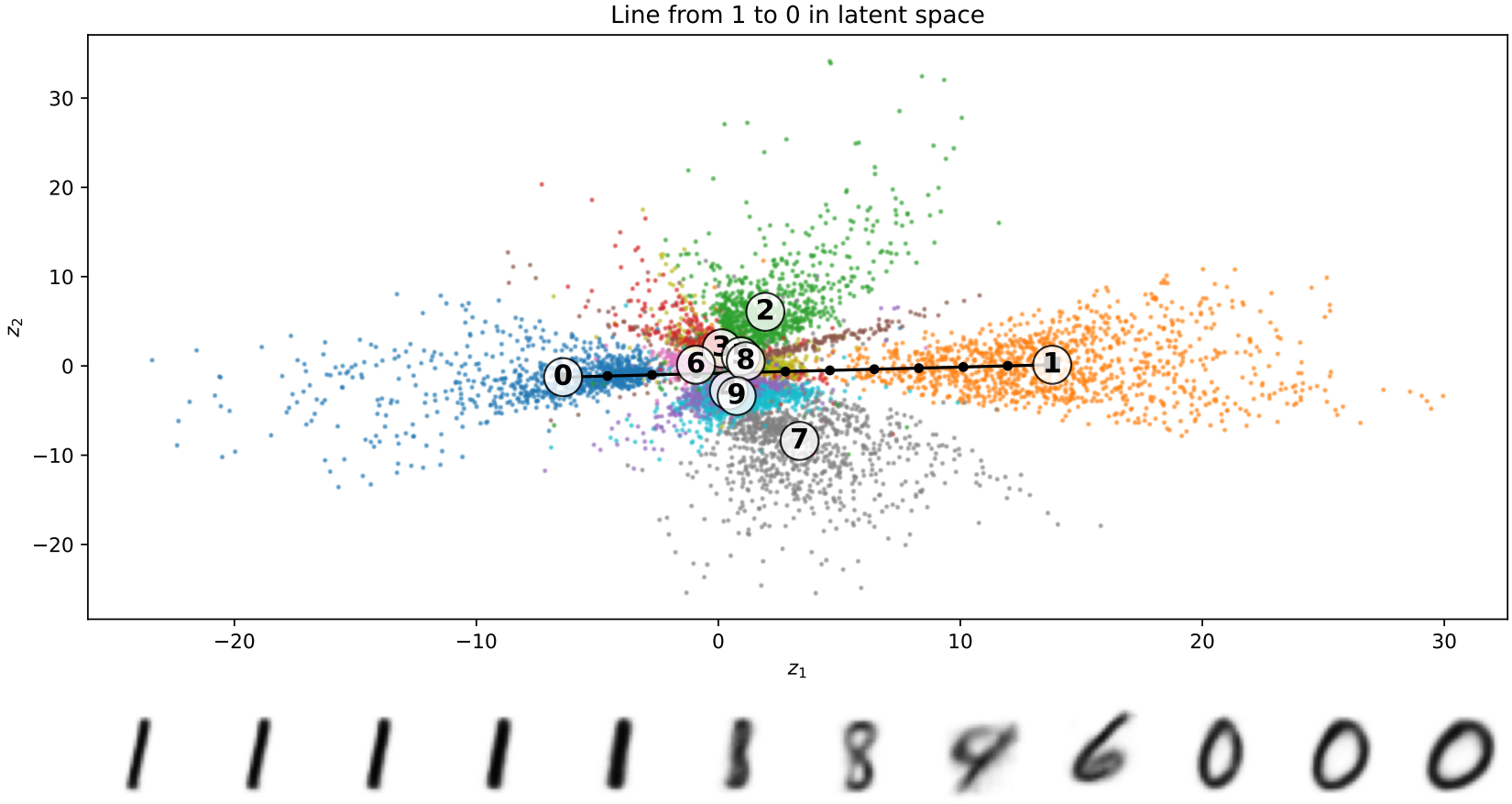


A line through latent space: 4 to 7



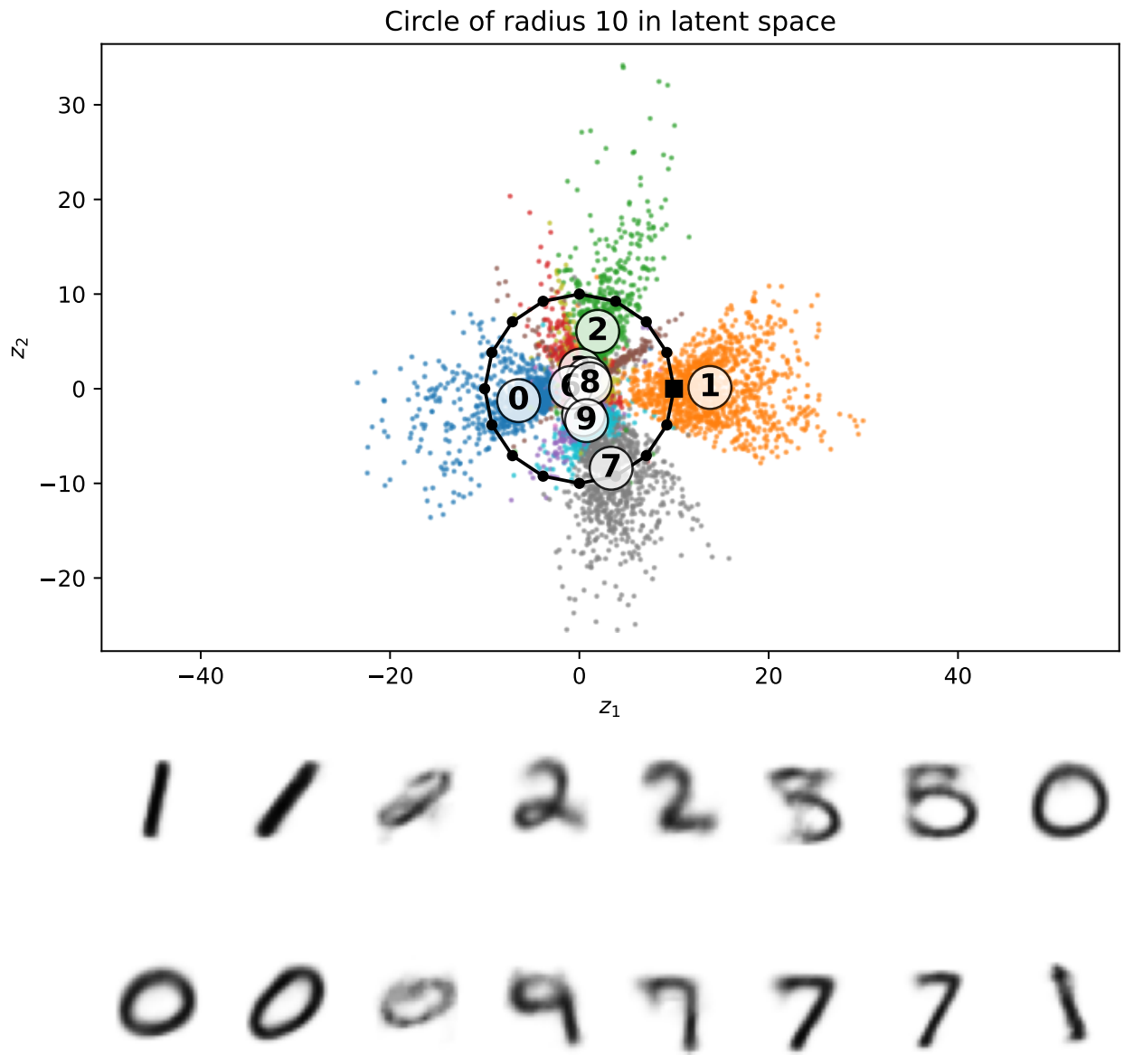
The 4 cluster lies underneath the 9 cluster, so the line starts by decoding 9s: in two dimensions the decoder cannot tell them apart and gives the more typical digit.

A line through latent space: 1 to 0



A straight line in latent space is not a straight path between digits: crossing the crowded middle we pass through 8, 4 or 9, and 6 on the way.

A circle in latent space



Relationship with PCA

Auto-encoder:

$$\text{original} \rightarrow \text{latent} \rightarrow \text{reconstruction}$$

$$\mathbf{x} \rightarrow \mathbf{z} = f(\mathbf{x}) \rightarrow \hat{\mathbf{x}} = g(\mathbf{z})$$

PCA:

$$\text{original} \rightarrow \text{PCA basis} \rightarrow \text{top PCs}$$

$$\mathbf{x} \rightarrow U^T\mathbf{x} \rightarrow (\mathbf{u}_1^T\mathbf{x}, \ldots, \mathbf{u}_k^T\mathbf{x})$$

In both cases the data are summarised by k numbers; for a *linear* auto-encoder these turn out to be the same summary.

The linear auto-encoder

Take centred data $\mathbf{x} \in \mathbb{R}^n$ with covariance

$$C = \frac{1}{N} \sum_a \mathbf{x}_a \mathbf{x}_a^T = U \Lambda U^T, \qquad \lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n.$$

A linear auto-encoder has $\mathbf{z} = W\mathbf{x}$ and $\hat{\mathbf{x}} = V\mathbf{z}$, with W a $k \times n$ matrix and V an $n \times k$ matrix, and squared-error loss

$$L = \frac{1}{N} \sum_a \|\mathbf{x}_a - VW\mathbf{x}_a\|^2.$$

Result (Baldi and Hornik, 1989): L is minimised when

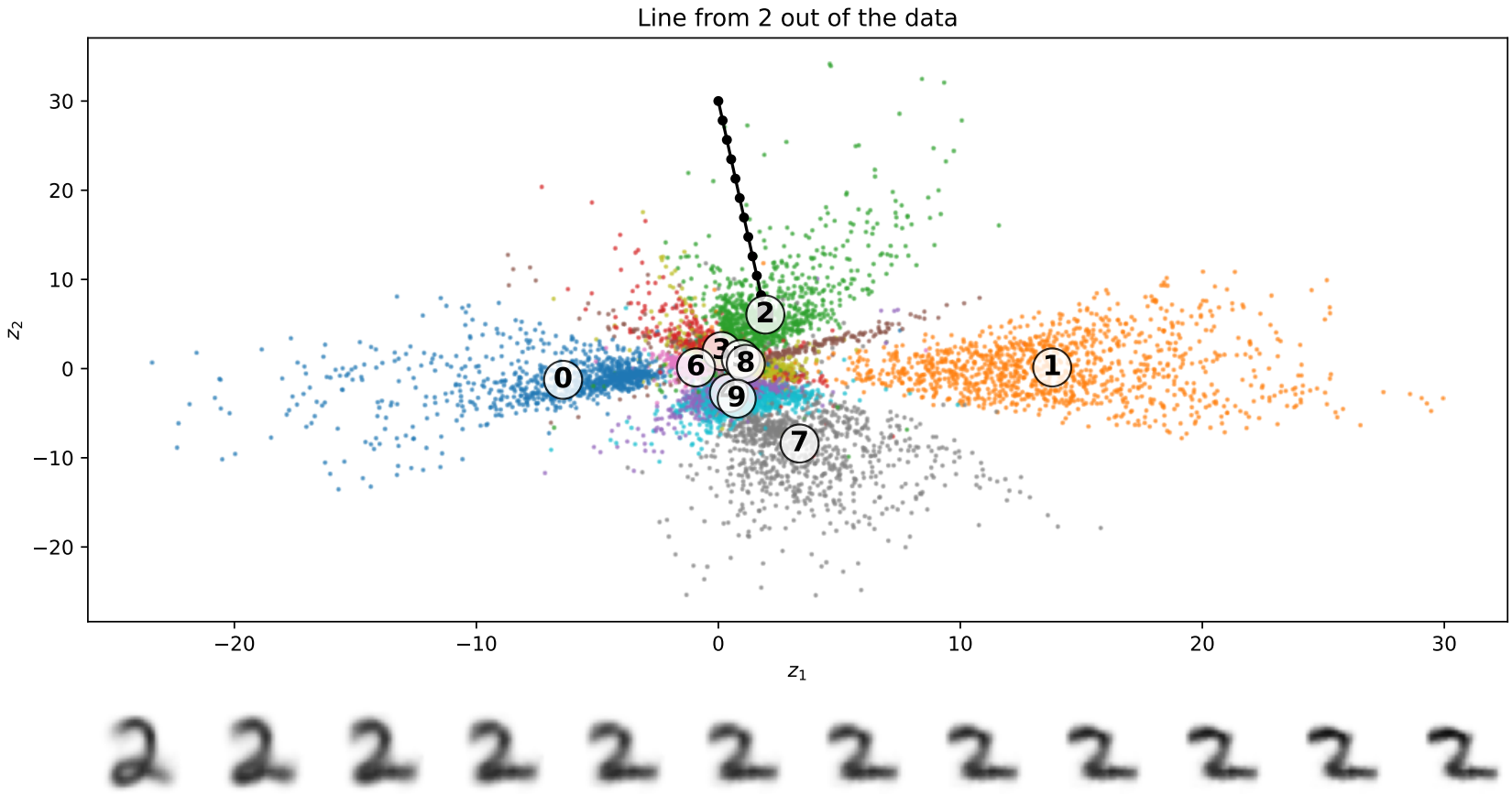
$$VW = U_k U_k^T,$$

the projection onto the top k principal components, and then $L = \lambda_{k+1} + \cdots + \lambda_n$.

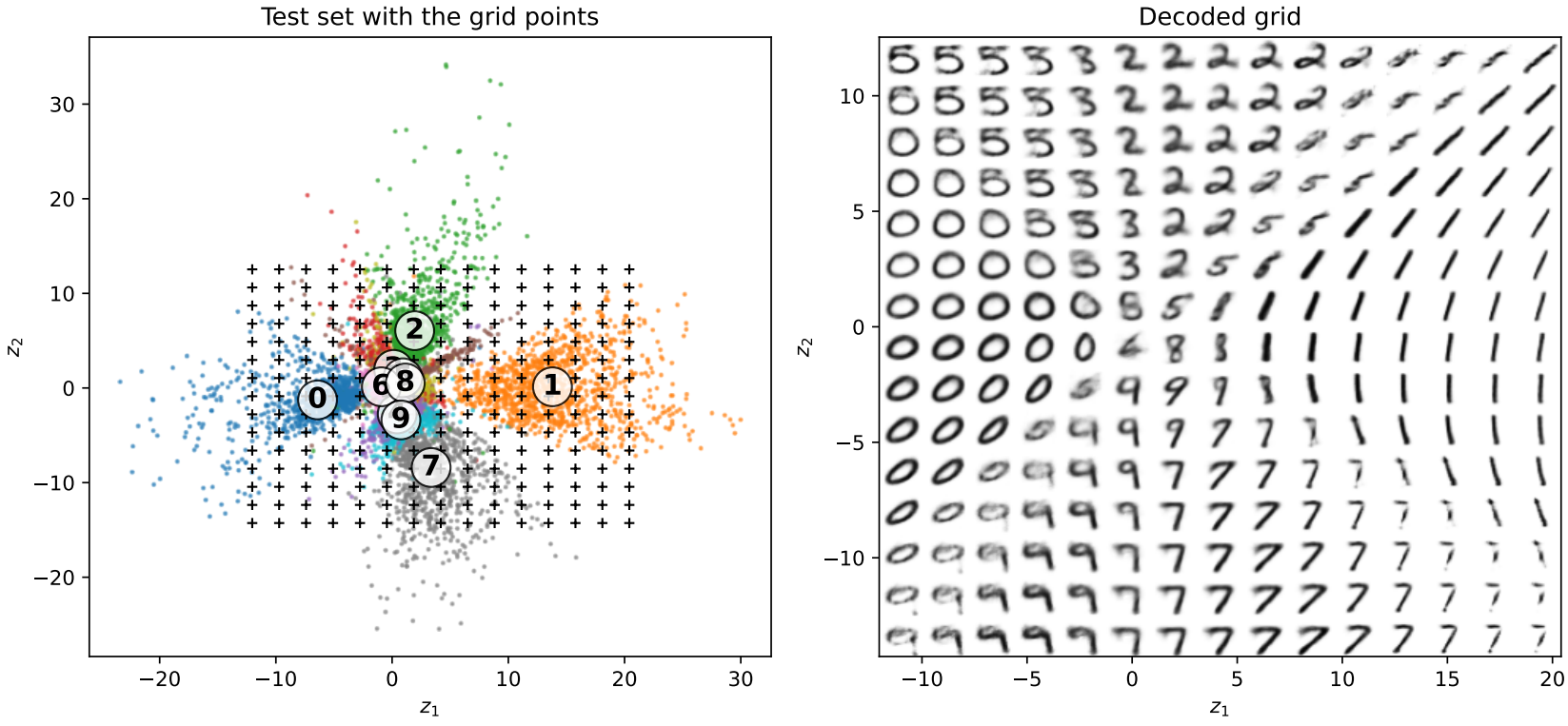
Beyond PCA

A nonlinear auto-encoder drops the restriction to a flat subspace and can follow a curved one.

Leaving the data



Decoding a grid over the latent space



So

This gives a latent space, but not a generative model.

Generative model

A *generative model* is a probability distribution over the data that we can sample from, so it can produce new examples that look like the training data.

The variational auto-encoder: the engineering idea

original $\rightarrow$ latent $\rightarrow$ latent $+$ noise $\rightarrow$ reconstruction

Adding noise to the latent representation means nearby points must decode to similar images, so the gaps get filled in, 'stitching together' the representation.

The variational auto-encoder: the engineering idea

original $\rightarrow$ latent $\rightarrow$ latent $+$ noise $\rightarrow$ reconstruction

However the network could cheat by spreading the latent points far apart so the noise doesn't matter; to stop this we also pull the latent points towards the origin.

The variational auto-encoder: formally

$$\mathbf{x} \to \begin{array}{l} \boldsymbol{\mu} = f_\mu(\mathbf{x}) \\ \boldsymbol{\sigma} = f_\sigma(\mathbf{x}) \end{array} \to z_i \sim \text{Normal}(\mu_i, \sigma_i) \to \hat{\mathbf{x}} = g(\mathbf{z})$$

The loss is

$$L = \text{reconstruction error} + \sum_i \mathrm{KL}\big(\text{Normal}(\mu_i, \sigma_i) \,\|\, \text{Normal}(0, 1)\big),$$

where the KL term is what pulls the latent points towards the origin.

Technical aside: the reparameterisation trick

We cannot backpropagate through a random sample, so instead we write

$$z_i = \mu_i + \sigma_i \xi_i, \qquad \xi_i \sim \text{Normal}(0, 1).$$

The randomness is now in ξ_i, which does not depend on the network, and z_i is a differentiable function of μ_i and σ_i.

The generative model

*T*hink of the decoder as a story about how a digit is made:

$$z_i \sim \text{Normal}(0, 1) \rightarrow x_j \sim \text{Bernoulli}\big(g_j(\mathbf{z})\big)$$

where $j \in \{1, 2, \ldots, 784\}$ labels pixels and x_j is the pixel value.

- **F**irst pick a point in latent space from a fixed, simple distribution, the *prior* $p(\mathbf{z})$.
- **T**hen the decoder gives a probability for each pixel being on, the *likelihood* $p(\mathbf{x}|\mathbf{z})$.

The generative model

*T*hink of the decoder as a story about how a digit is made:

$$z_i \sim \text{Normal}(0, 1) \rightarrow x_j \sim \text{Bernoulli}\big(g_j(\mathbf{z})\big)$$

It is like $\mathbf{z}$ is the idea and it has been 'disguised' as an image.

The encoder

Given a digit $\mathbf{x}$, which $\mathbf{z}$ produced it? That is the *posterior*.

$$p(\mathbf{z}|\mathbf{x}) = \frac{p(\mathbf{x}|\mathbf{z})\, p(\mathbf{z})}{p(\mathbf{x})},$$

but $p(\mathbf{x}) = \int p(\mathbf{x}|\mathbf{z}) p(\mathbf{z})\, d\mathbf{z}$ is intractable.

So we approximate it:

$$q(\mathbf{z}|\mathbf{x}): \quad z_i \sim \text{Normal}(\mu_i, \sigma_i), \qquad \boldsymbol{\mu} = f_\mu(\mathbf{x}),\ \boldsymbol{\sigma} = f_\sigma(\mathbf{x}).$$

The encoder is *amortised* inference: one network gives the approximate posterior for every $\mathbf{x}$.

The encoder

$$q(\mathbf{z}|\mathbf{x}): \quad z_i \sim \text{Normal}(\mu_i, \sigma_i), \qquad \boldsymbol{\mu} = f_\mu(\mathbf{x}),\ \boldsymbol{\sigma} = f_\sigma(\mathbf{x}).$$

We are following the general deep learning philosophy, if there is some function we don't know we make a function for it.

Where the loss comes from

We want to maximise $\log p(\mathbf{x})$ over the training data: this is maximum likelihood, and it fits the model's distribution to the data's.

Where the loss comes from

Since $p(\mathbf{x}) = p(\mathbf{x}, \mathbf{z})/p(\mathbf{z}|\mathbf{x})$ for any $\mathbf{z}$, we can average over $\mathbf{z} \sim q(\mathbf{z}|\mathbf{x})$ and multiply and divide by q:

$$\log p(\mathbf{x}) = \mathbb{E}_q \left[\log \frac{p(\mathbf{x}, \mathbf{z})}{q(\mathbf{z}|\mathbf{x})} \right] + \mathbb{E}_q \left[\log \frac{q(\mathbf{z}|\mathbf{x})}{p(\mathbf{z}|\mathbf{x})} \right].$$

Where the loss comes from

A bit of algebra gives

$$\log p(\mathbf{x}) = \underbrace{\mathbb{E}_q\big[\log p(\mathbf{x}|\mathbf{z})\big] - \mathrm{KL}\big(q(\mathbf{z}|\mathbf{x}) \,\|\, p(\mathbf{z})\big)}_{\text{ELBO (evidence lower bound)}} + \mathrm{KL}\big(q(\mathbf{z}|\mathbf{x}) \,\|\, p(\mathbf{z}|\mathbf{x})\big).$$

- **T**he last KL is non-negative, so the ELBO is a lower bound on $\log p(\mathbf{x})$, and maximising it improves both the model and the approximation q.
- **T**he first term is minus the binary cross-entropy: the reconstruction error.
- **T**he second is the KL to Normal$(0, 1)$.

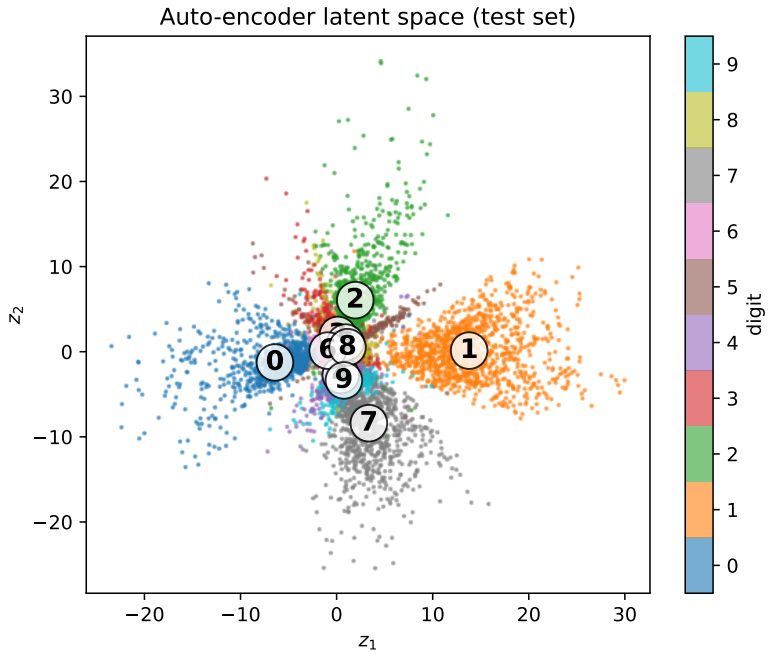
So the loss we guessed on engineering grounds is $L = -\text{ELBO}$.

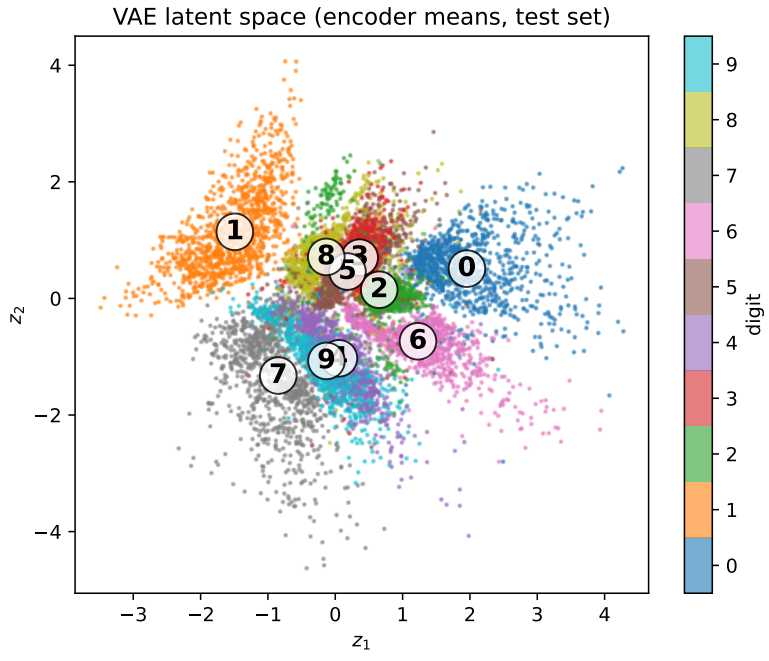
A VAE for MNIST

$$784 \to 512 \to 256 \to \begin{matrix} \boldsymbol{\mu} \in \mathbb{R}^2 \\ \log \boldsymbol{\sigma}^2 \in \mathbb{R}^2 \end{matrix} \to \mathbf{z} \in \mathbb{R}^2 \to 256 \to 512 \to 784$$

Same layers as before; the encoder outputs $\log \boldsymbol{\sigma}^2$ rather than $\boldsymbol{\sigma}$ so it does not need to be kept positive.

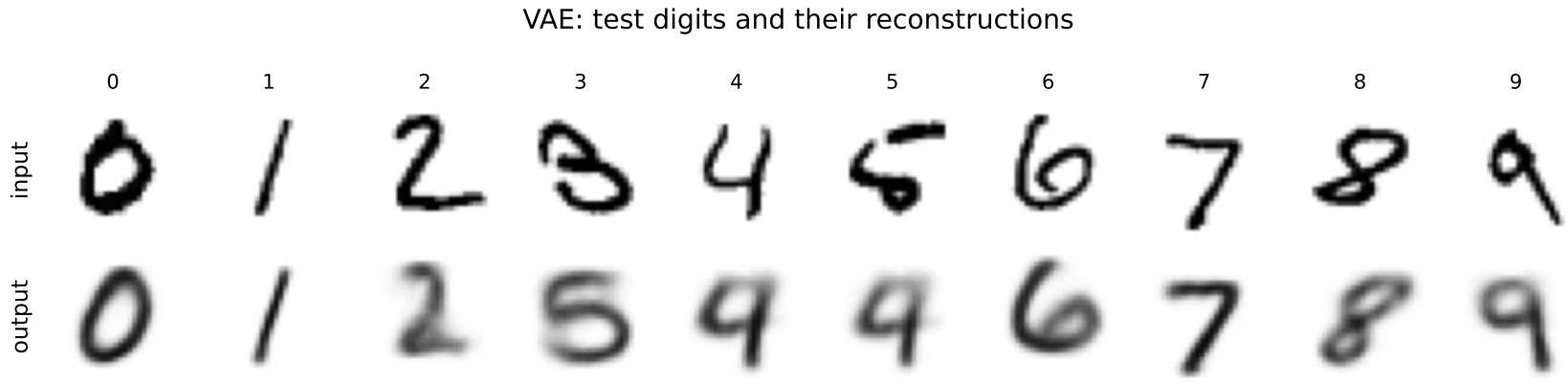
Auto-encoder and VAE latent spaces





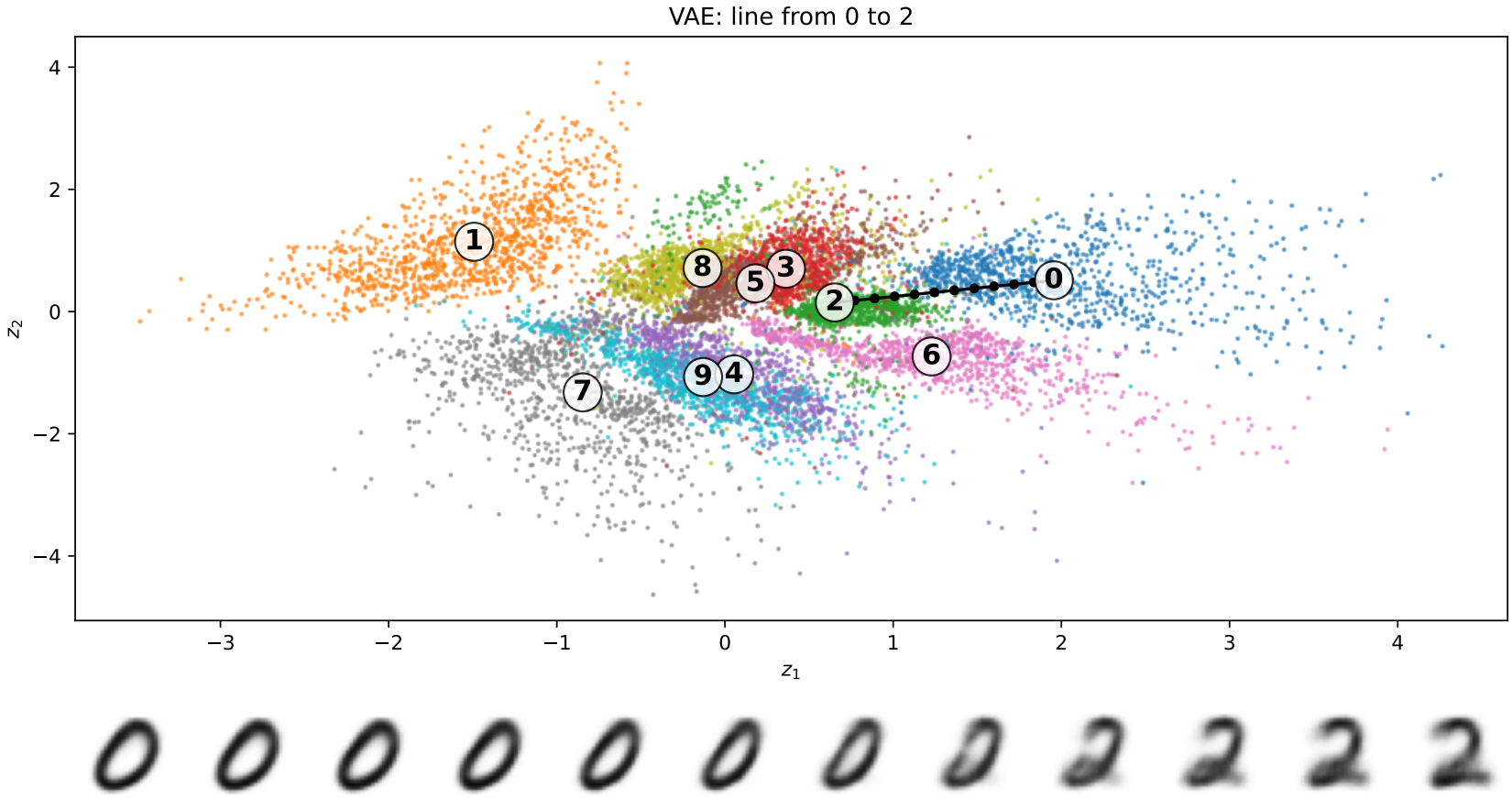
The VAE latent space is compact, centred on the origin and has no gaps between the clusters.

VAE reconstructions



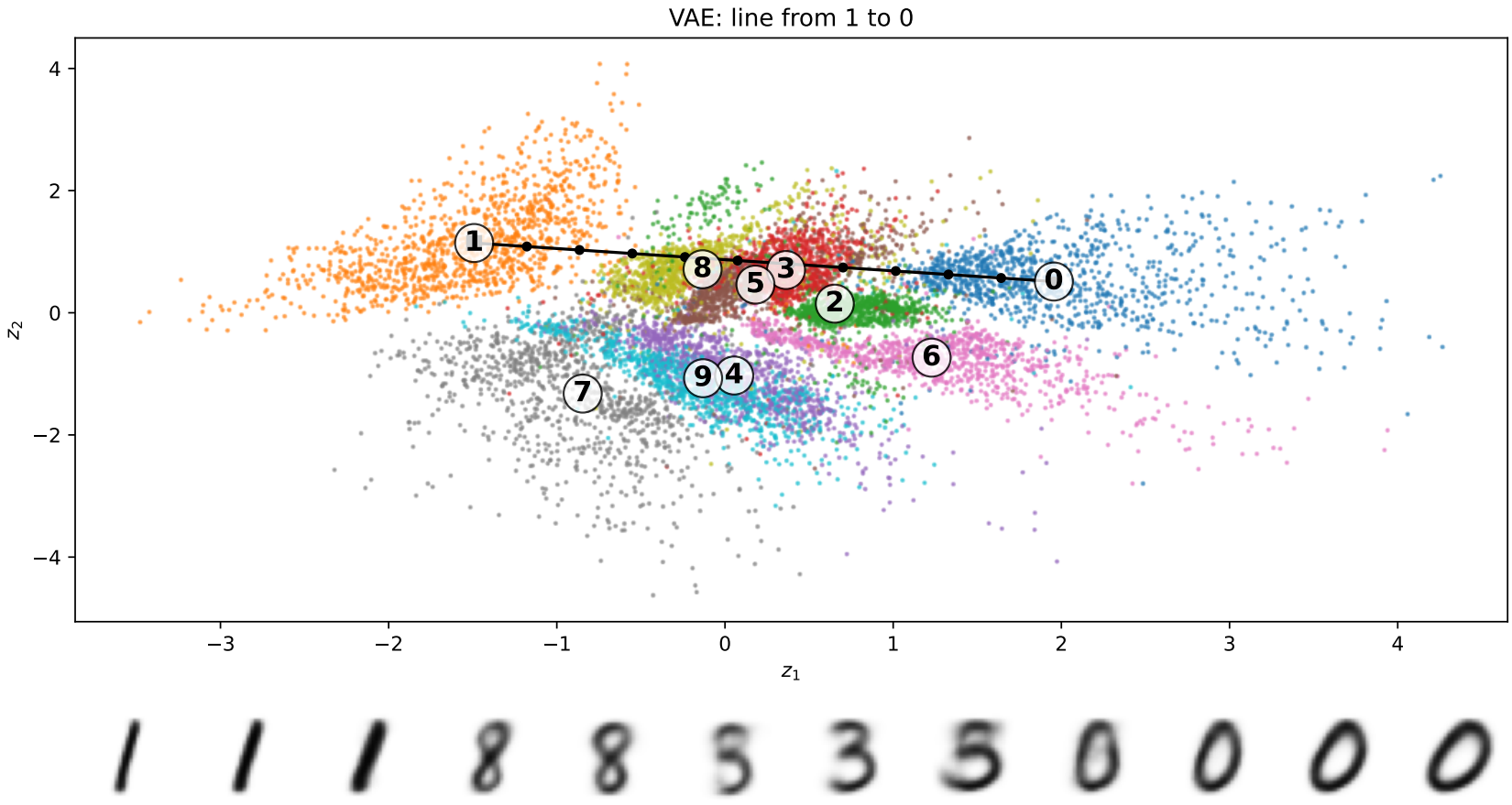
A little blurrier than the auto-encoder: the price of the KL term.

VAE: a line from 0 to 2

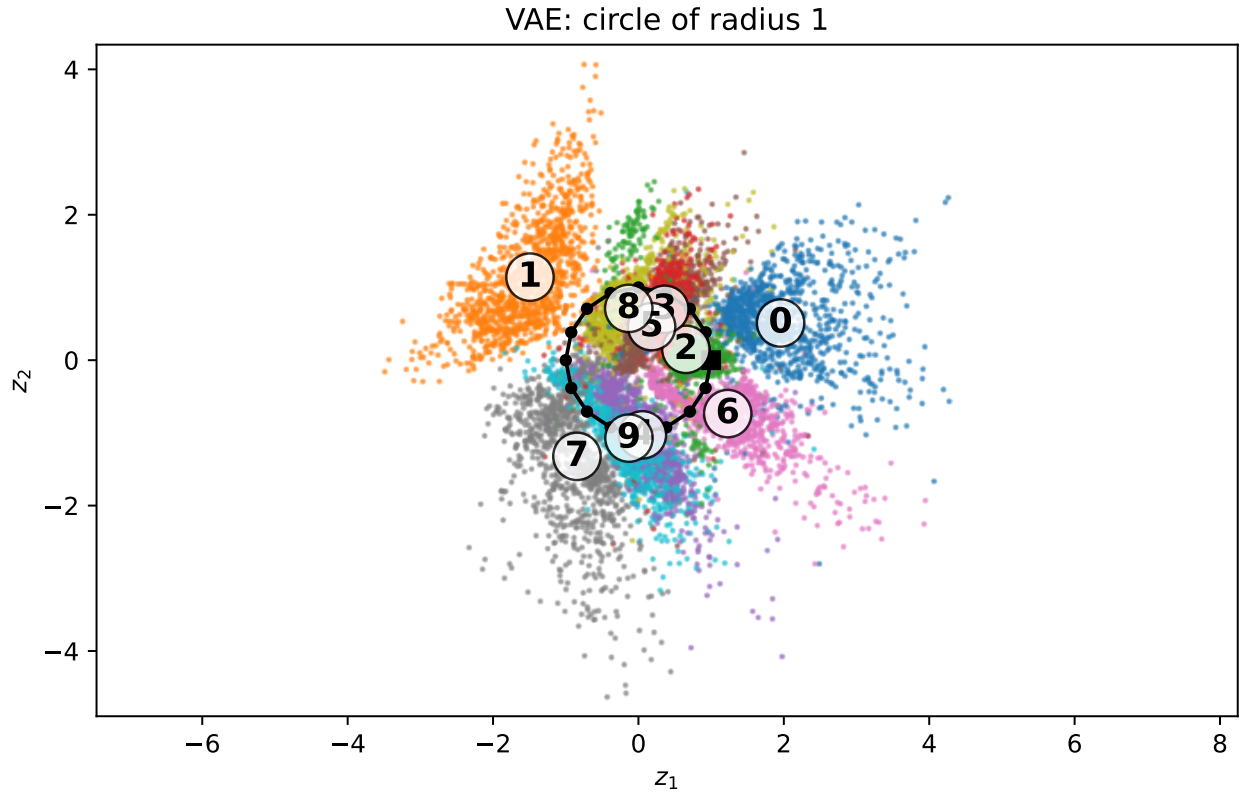


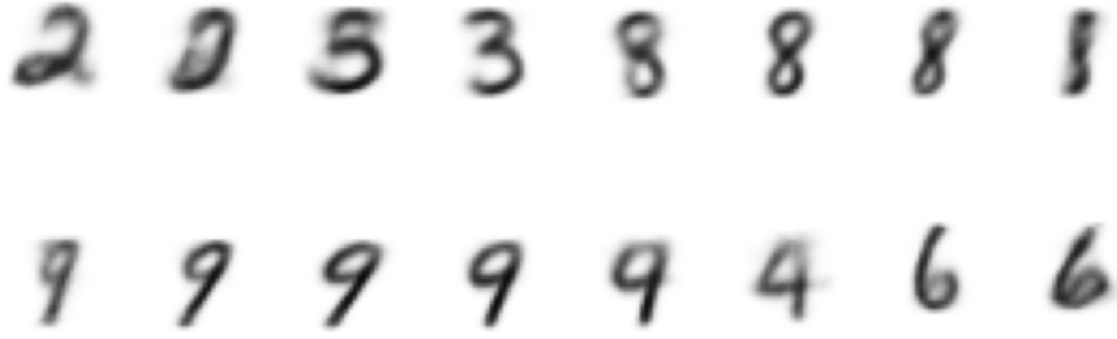
Compare with the auto-encoder: the path between clusters now decodes to digits throughout.

VAE: a line from 1 to 0

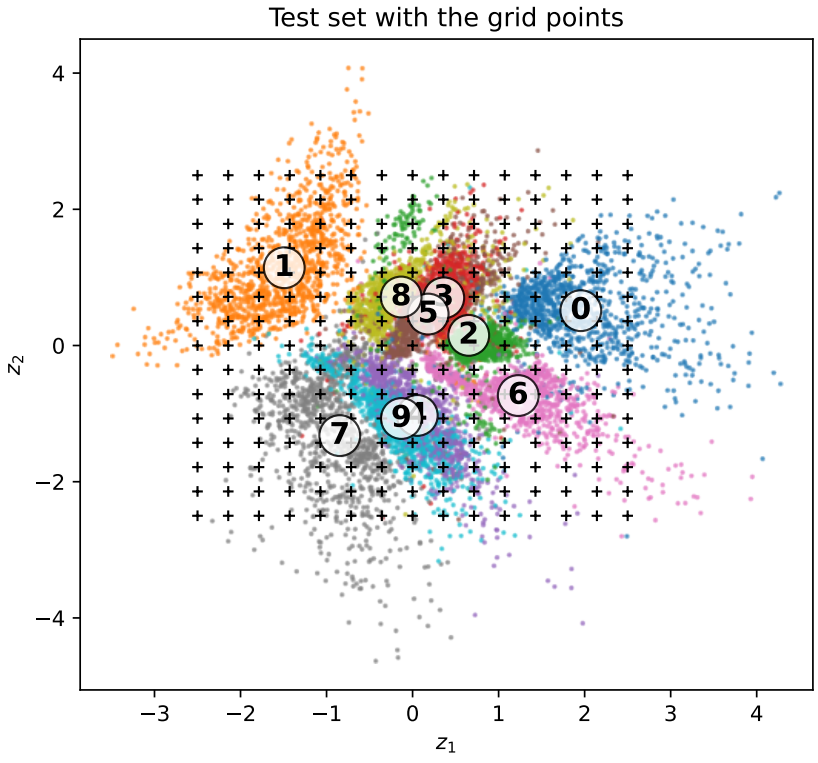


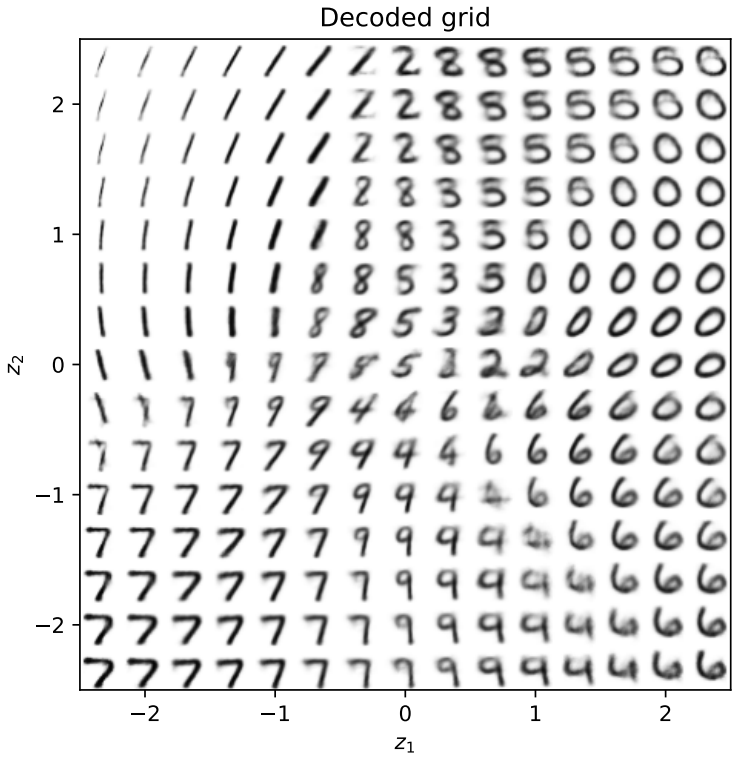
VAE: a circle in latent space





VAE: decoding a grid over the latent space





New digits

$$z_i \sim \text{Normal}(0, 1) \rightarrow \hat{\mathbf{x}} = g(\mathbf{z})$$

New digits: $z_i \sim$ Normal(0,1), decoded

